

# Dependency Injection And Strategy Design Pattern

console debug:container => get all services registered within the container

All services in container has a **unique** identifier , and the default services.yaml configuration, automatically register services by giving the FQCN to each service as its ID

When ID is equal to FQCD, it is optional to define class in services.yaml since it is done automatically

when you define an alias to some class, it is counted a new service along with the default loaded one.

```
services:
  _defaults:
    autowire: true
    autoconfigure: true
    bind:
      string $booo: "Hello i'm boo from defaults"
      App\Services\cart\discount\DiscountInterface $dis: "@regular.discount"
      App\Services\Boo $boo: '@App\Services\Boo'

  App\: <=====
    resource: "../src/" <=====
```

This load the service by default by its FQCN as its Id (app\: resources: ../src/), so here we have tree roads:

1. what happen when we explicitly define the service like:

```
App\Services\Boo:
  class: App\Services\Boo
  arguments:
    $booo: "Hey i'm boo"
```

This does **NOT** create a second service.

It **overrides (replaces)** the auto-registered definition.

Symfony merges definitions by **service ID**.

Since the ID is the same (^App\Services\Boo`), the explicit definition wins.

So:

■ Not duplicated

■ Same service ID

■ Manual config overrides auto one

2. what about defining an alias as value for a service:

```
App\Contracts\FooInterface: '@App\Services\Boo'
```

Here, is **not creating a new service**. It is an **alias definition**; Even though you use it mainly to control what gets injected when you type-hint the interface, technically in Symfony this is still an **alias** used to

point to the service that will be used when the user type hints *App\Contracts\FooInterface*

3. what will happen when we define an **alias**:

```
regular.discount:
  class: App\Services\cart\discount\RegularDiscount
  arguments:
    $discount: 0.0
```

This creates a **new service** with ID: ``regular.discount``

This is important because you also have:

```
App\:
  resource: "../src/"
```

Symfony already auto-registered: ``App\Services\cart\discount\RegularDiscount``

So now you have:

1. ``App\Services\cart\discount\RegularDiscount`` (auto-loaded)
2. ``regular.discount`` (manual definition)

■ These are **two different services**, even though they use the same class and they are completely separate definitions.

### ## PLEASE NOTE THAT:

First, this YAML: ``alias: class: ...`` is **NOT an alias** in Symfony ■■.

That is a **new service definition** with ID ``alias``.

An actual alias looks like this: ``some.id: '@other.service`` (No ``class:`` allowed.)

This is also a way to define an alias:

```
foo:
  alias: App\Services\Boo
```

### ### Private vs Public services:

Please notice that symfony by default mark all services as private services, to encourage use autowiring and prevent you from fetching services manually from the container

If services are public you can use get method of container to use services in container manually, and pay attention that public services are instantiated into container regardless of whether you use them or not (eager loaded).

In the opposite side, private services if you don't use them, they will not be instantiated within the container (they are lazy loaded)

#### #### ■ Private service (default)

```
App\Service\Foo:  
    public: false
```

- Can be autowired
- Can be injected
- ■ Cannot be fetched with ``$container->get()``
- Can even be optimized away if unused

#### #### ■ Public service

```
App\Service\Foo:  
    public: true
```

- Can be injected
- Can be fetched manually : ``$container->get(Foo::class);``
- Cannot be removed or inlined during container optimization

Private services allow Symfony to:

- Inline services
- Remove unused services
- Optimize container
- Compile a faster container

Public services must always remain accessible.

So private = better performance + cleaner architecture.

```
php bin/console debug:autowiring --all
```

- -all here means it shows **all autowireable types** , if you don't use it, symfony shows only the most relevant autowireable types (usually public & common ones).

---

In UML, to represent DI, use << uses >> within the arrow from the encubator to the encubated, like controller uses a service by injecting it in constructor, so uses and the arrow is from controller to the service, since the controller is the one who uses the service (by injecting it and using it)

---

To remove the container, you can clear cache: **rm -Rf /var/cache/\***

You can investigate /var/cache and search for service if it is private and never used, you will find that it is removed and the reason is: **unused**

---

===== STRATEGY =====

---

```
final class AdMessage { // <===== DTO
    public function __constructor(public string $to, public string $message) {
    }
}
```

within src/Advertiser/Strategy/SmsAdStrategy.php

```
final class SmsStrategy {
    public function advertise(AdMessage $adMessage) {}
}
```

Create a controller:

```
class AdController {
    public function __constructor(private SmsAdStrategy $strategy)

    public function index()
    {
        $ad = new AdMessage(message: "Hello sms world", to: "123456789");
        $this->strategy->advertise($ad);

        # render the view
    }
}
```

Within the controller, it has an SmsAdStrategy, so when container inject the controller, it find that the controller also needs an argument to be resolved, so it resolve the argument fdirst, then inject the container (symfony DI knows this cauz of autowiring)

The observation here is that we don't want to couple the controller with the strategy to avoid having a strong coupling, so we need an a **class-in-the-middle that resides in the middle** and control which strategy to be used. In this case the controller has a strong coupling with **SmsAdStrategy**. This way

the controller will not have any visibility for which way we want to advertise or which strategy we are using, it is the job of AdAdvertiser who decide which way will be used so let's continue:

So let's create the class: AdAdvertiser

```
class AdAdvertiser {
    public function __construct(private SmsAdStrategy $strategy){}

    public function send(AdMessage $message) {
        $this->strategy->advertise($message);
    }
}
```

Now lets modify the controller to use the Middle class instead of the strategy

```
class AdController {
    public function __constructor(private AdAdvertiser $advertiser)

    public function index()
    {
        $ad = new AdMessage(message: "Hello sms world", to: "123456789");
        $this->advertiser->send($ad);

        # render the view
    }
}
```

Now we can configure our AdAdvertiser using manual wiring like so:

```
# services.yaml
App\Advertiser\AdAdvertiser:
    arguments:
        $strategy: "@App\Advertiser\Strategy\SmsAdStrategy" # -> you can use alias @alias
```

Up until now, even though we make controller unaware about the strategy we used, the AdAdvertiser itself has a strong coupling with SmsAdStrategy, it is like we did nothing. To handle this, we can create an interface that all strategies will implement, then in the middle-class, **we make it depend on abstraction (on interface that all strategies implement)** (on interface that all strategies implement)

So in brief:

- We create an interface that all strategies will depend on
- Create strategies and make them implement the interface
- Make the middle-class depend on interface (abstraction -> Liskov + Dep-Inv)
- Control strategies from services.yaml easily

---

#### 1. Interface creation

```
interface AdStrategyInterface {  
    # this is called Functional interface (an interface that has one and only one method)  
    public function advertise(AdMessage $message): void;  
}
```

#### 2. Make strategies that implement the interface

```
final class SmsAdStrategy implements AdStrategyInterface {}
```

#### 3. Make Middle-class depend on abstraction instead of strategy

```
class AdAdvertiser {  
    public function __construct(AdStrategyInterface $strategy) {}  
}
```

4. Now, at this moment, the container does not know how to inject args typehinted with AdStrategyInterface, so we need to guide it by tuning the strategy in services.yaml

```
sms.strategy:  
    class: App\Advertiser\Strategy\SmsAdStrategy  
  
App\Advertiser\Strategy\AdStrategyInterface: "@sms.strategy"  
  
# we can define alias by going above strategy class and add:  
#[Alias("sms.strategy")]
```

**Remember:** If you have only one service that implement an interface, Symfony automatically created an alias by mapping the service to the interface, so when you type-hint the interface the service get used automatically without extra configuring or defining the service in services.yaml

**BUT:** If you have many services that implements the interface, Symfony will get confused and don't know which one to use when the interface get type-hinted, so in this case you must explicitly tell symfony which one by defining it in services.yaml

#### ## Important notes on aliasing:

You can do it like this one:

```
App\Advertiser\Strategy\AdStrategyInterface:
    alias: App\Advertiser\Strategy\SmsAdStrategy
```

You can use named aliases: (you work with type AND also with name of argument)

Like when you use `LoggerInterface $cacheLogger`, will use a cache logger, but `LoggerInterface $mailerLogger` will use a different logger, **depends on the name** of argument you specify

```
public function __construct(private LoggerInterface $cacheLogger) {}
public function __construct(private LoggerInterface $mailerLogger) {}
```

Do the following to take a look at services: **console debug:autowiring Logger**

In our example:

```
App\Advertiser\Strategy\AdStrategyInterface $whatsappStrategy: '@whatsapp.strategy'

# Here you must specify one strategy to be used as default when the above name is not
# type-hinted with (choose a strategy and put #[AsAlias] above the class)
```

You can also use `#[Autowire]` as following

```
public function __construct(
    #[Autowire(service: WhatsappStrategy::class)] private AdStrategyInterface $strategy)
```

You can also use named aliases using `#[Target]`

```
public function __construct(
    #[Target(name "whatsappStrategy")] private AdStrategyInterface $strategy)
```

Using target is preferable when you have multiple services that match an interface id, and symfony does not know which to inject.

## ■ The Core Difference

`#[Autowire]` ===== `#[Target]`

---

Inject a specific value..... Select which service ID to use

Works for parameters, services, expressions.....Only used for service disambiguation

Overrides autowiring logic.....Helps autowiring choose

You can even inject a parameter:

```
public function __construct(#[Autowire(param: "app.website")] private string $websiter)
# or (the above is explicit and clear to read)
public function __construct(#[Autowire("%app.website%")] private string $websiter)
```